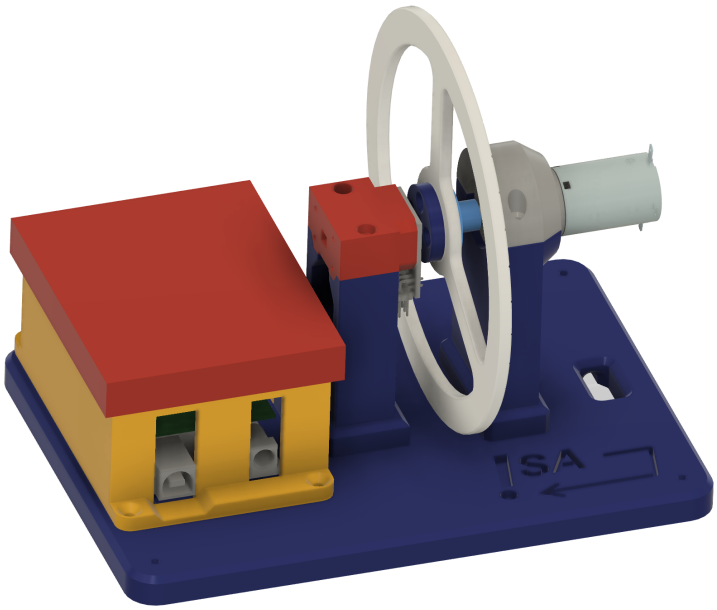
User Manual

DC Motor Control Laboratory Equipment



Department of Systems Engineering and Automation (ISA)

Universidad de Sevilla · ialvarado@us.es

November 7, 2026

What this equipment does: a Maxon DC motor with a 14-bit magnetic encoder, controlled in closed loop from MATLAB/Simulink via an Arduino. The control loop *runs entirely on the PC*: the Arduino only reads the encoder and applies the PWM commanded by MATLAB every **10 ms** (100 Hz). Communication is via USB/serial at 115 200 baud.

What the instructor needs: connect the USB cable, select the serial port in Simulink, and start the experiment. Everything else is automatic.

Contents

1 Equipment Description **3**
1.1 Main Components . . . 3
1.2 Connection Diagram . . . 3
1.3 System Parameters . . . 4

2 Initial Setup (one-time only) **4**
2.1 Install Arduino Libraries . . . 4
2.2 Upload Firmware to the Arduino . . . 4
2.3 Check Shield Wiring . . . 5

3 Usage in Each Lab Session **5**
3.1 Step 1 — Connect the Hardware . . . 5
3.2 Step 2 — Select the Serial Port in Simulink . . . 6
3.3 Step 3 — Launch the Experiment from Simulink . . . 6
3.4 Step 4 — Save Experiment Data . . . 7

4 Quick Reference **7**
4.1 Serial Communication Protocol . . . 7
4.2 Arduino / Shield Pin Assignments . . . 7
4.3 Control Signal to Average Motor Voltage . . . 8

5 Troubleshooting **8**

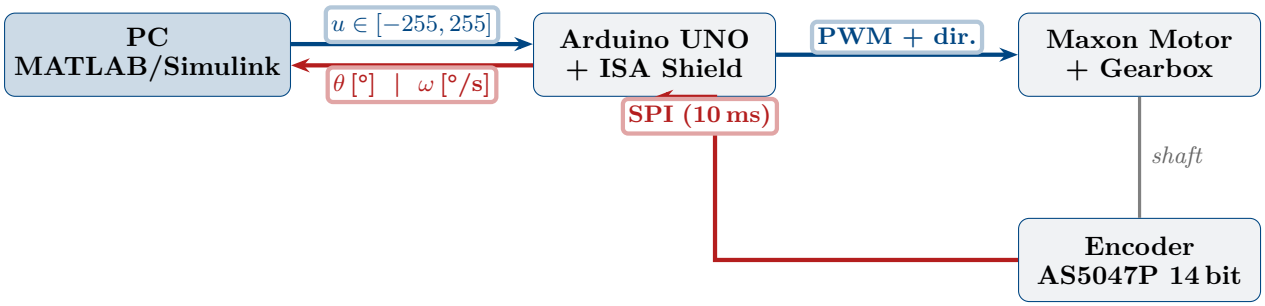
6 Repository and Resources **9**

1 Equipment Description

1.1 Main Components

Component	Model	Function
Motor	Maxon A-max 22	Precision DC motor with GP 22 B planetary gearbox
Encoder	ams AS5047P	14-bit magnetic encoder (16 384 steps/rev), load-side mounted
Driver	TB6612FNG	Dual motor driver integrated on the shield PCB
MCU	Arduino UNO	Reads encoder, applies PWM, communicates with MATLAB via USB
Shield PCB	ISA custom	Stacks directly on the Arduino; integrates driver + encoder connector

1.2 Connection Diagram



⟶ Control signal ⟶ System measurements — Mechanical coupling

Note

The complete control loop runs in MATLAB. The Arduino acts as a **real-time hardware interface**: it sets the experiment cadence via Timer2 interrupts every **10 ms** (100 Hz), reads the encoder over SPI, computes the numerical velocity ($\Delta\theta/T_s$), and applies to the driver the control signal received from MATLAB.

Angle accumulation: the AS5047P encoder outputs the angle in the range $[0°, 360°)$ per revolution. The firmware counts complete revolutions and accumulates the angle, so the output is an unbounded angle $\in (-\infty, +\infty)$ that grows (or decreases) continuously without jumps or resets — just like an incremental encoder.

Luenberger observer (disabled by default): estimates filtered position and velocity instead of the raw numerical derivative. Enable it by uncommenting four lines at the end of the transmit block in `main.ino`.

1.3 System Parameters

Parameter	Value	Notes
Sampling period T_s	10 ms (100 Hz)	Fixed in Arduino firmware
Communication speed	115 200 baud	USB/serial
Control signal range	$u \in [-255, 255]$	Maps to $\pm V_{\text{supply}}$ (see note)
Encoder resolution	$\approx 0.022°$/step	16 384 steps/rev
Approximate model	$G(s) = K/(\tau s + 1)$	K and τ depend on V_{supply}

△ Supply Voltage

The signal $u \in [-255, 255]$ maps to $\pm V_{\text{supply}}$, which depends on how the equipment is powered:

- **External supply (recommended):** max. **9 V**; in practice **8.4 V** is recommended to avoid excessive motor heating. $u = 255 \Rightarrow 8.4\,\text{V}$.
- **USB only (no external supply):** the motor runs at $\approx \pm 5\,\text{V}$. Reduced speed range, but fully functional for initial tests and sessions without a bench power supply.

The model $G(s) = K/(\tau s + 1)$ must be identified experimentally for each supply voltage used.

2 Initial Setup (one-time only)

This section is only needed the **first time** the equipment is configured, or if the Arduino is replaced. It does not need to be repeated each lab session.

2.1 Install Arduino Libraries

1. Open **Arduino IDE 2.x**.
2. Go to `Tools → Library Manager`.
3. Search for and install `AS5047P` (author: Marginally Clever Robots).
4. Search for and install `MsTimer2` (author: Javier Valencia).

■ Note

These libraries only need to be installed once on the lab PC. If a different computer is used, repeat this step.

2.2 Upload Firmware to the Arduino

1. Connect the Arduino to the PC via USB.
2. Open the file `firmware/arduino/main/main.ino` from the repository.
3. Select the correct board: `Tools → Board → Arduino UNO`.
4. Select the port: `Tools → Port → COMx` (Windows) or `/dev/tty.usbmodem...` (Mac/Linux).

5. Click the ↑ **Upload** button (or press `Ctrl+U`).
6. Wait for the message *"Upload successful"* to appear.

✔ Success

To verify the firmware is running: open the **Serial Monitor** in the IDE (`Ctrl+Shift+M`) and set it to `115200 baud`. You will see **garbled characters or meaningless symbols** scrolling at high speed — this is completely normal. The data travels binary-encoded, not as readable text, so the Serial Monitor displays it as gibberish. The important thing is that *something appears*: if the screen stays blank, check the encoder wiring.

△ Warning

The firmware is already loaded on the lab equipment. Only re-upload it if the Arduino is replaced or the firmware is updated.

2.3 Check Shield Wiring

The shield PCB stacks directly on the Arduino (no cables needed). Verify that:

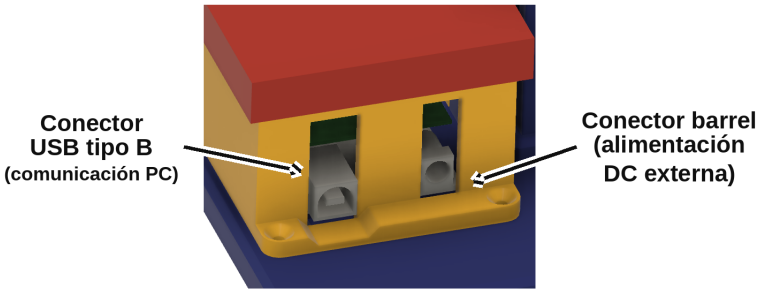
- The shield is fully seated on the Arduino (all pins engaged).
- The motor cable is connected to the driver terminals.
- The encoder cable (7-pin JST connector) is connected to the shield.
- The power supply is connected to the Arduino's power jack (recommended: **8.4 V**; max. 9 V). Without an external supply, the equipment runs on USB alone at ≈ 5 V.

3 Usage in Each Lab Session

This is the standard procedure at the start of each session. With the equipment already configured, it takes less than two minutes.

3.1 Step 1 — Connect the Hardware

1. Connect the Arduino USB cable to the PC.
2. **Optional:** connect the power supply to the Arduino's barrel connector (recommended **8.4 V**, max. 9 V). Without an external supply, the equipment works on USB alone (≈ 5 V) with a reduced speed range.



Front panel detail. Left: USB type-B connector (PC communication). Right: ***barrel connector*** *(external DC power input, 8.4 V recommended).*

3. The Arduino LED should turn on. ✔

△ Warning

Always connect the USB first, then the external power supply. When disconnecting, reverse the order: power supply first, then USB.

3.2 Step 2 — Select the Serial Port in Simulink

The port is configured directly inside the Simulink model — no additional MATLAB commands are needed.

1. Open the model `software/matlab/MotorCC_V3.slx`.
2. **Double-click** the central block **"Motor CC"**. A parameter window will open showing a drop-down list of the serial ports currently available on the PC.
3. Select the port that corresponds to the Arduino and click **OK**.

■ Which port is the Arduino?

If it is not obvious which port belongs to the Arduino:

1. **Disconnect** the Arduino USB cable.
2. Double-click the **"Motor CC"** block and note the ports in the list.
3. **Reconnect** the Arduino USB cable.
4. Double-click the block again: the new port that now appears is the Arduino. Select it and click OK.

3.3 Step 3 — Launch the Experiment from Simulink

1. With the port already selected in the **"Motor CC"** block (Step 2), press the **Run** button (▶) in the Simulink toolbar.
2. The model connects automatically to the Arduino and starts the control loop at 100 Hz.
3. To stop the experiment, press the **Stop** button.

△ **Stop the simulation only when the motor is at rest**

Stopping the simulation (pressing **Stop**) *while the motor is running* can be hazardous: when communication is lost, the driver holds the **last PWM value** it received indefinitely, and the motor keeps spinning without control. Always wait until the motor is **stopped** (control signal $u = 0$) before pressing Stop, or bring u to zero from Simulink before stopping.

✔ **Success**

If everything is working, the Simulink scope will show the angular position (in degrees) and the applied control signal in real time. The motor will respond to the configured reference.

3.4 Step 4 — Save Experiment Data

The model `MotorCC_V3.slx` does not include any data-saving blocks: each student is expected to add the necessary blocks (*To Workspace*, *To File*, *Scope* with auto-save, etc.) to their own model, following the instructions for each lab assignment.

■ **Note**

The variables available each Simulink cycle are: angular position (°), angular velocity (°/s), and control signal $u \in [-255, 255]$. Refer to each lab guide for the exact blocks to add and the variable names to use for export.

4 Quick Reference

4.1 Serial Communication Protocol

Direction	Binary format and meaning
Arduino → MATLAB	`'V'` | `float(4B)` | `float(4B)` | `float(4B)` | `'\n'` *angle [°] · velocity [°/s] · applied PWM*
MATLAB → Arduino	`'A'` | `int16(2B)` | `'\n'` *control signal* $u \in [-255, 255]$

Transmission rate: **100 Hz** ($T_s = 10$ ms). Binary format (little-endian).

4.2 Arduino / Shield Pin Assignments

Arduino Pin	Signal	Description
4	CS (SPI)	Chip Select — AS5047P encoder
11	MOSI	SPI MOSI
12	MISO	SPI MISO
13	CLK	SPI Clock
9	EnA (PWM)	Enable / PWM — driver channel A
7	Ain1	Motor direction (bit 1)
8	Ain2	Motor direction (bit 2)

4.3 Control Signal to Average Motor Voltage

The TB6612FNG driver switches the supply voltage via PWM. The **average voltage** applied to the motor is proportional to the duty cycle:

$$\overline{V}_{\text{motor}} = \frac{u}{255} \cdot V_{\text{supply}}$$

where V_{supply} is the driver supply voltage (**8.4 V** recommended with an external supply; **≈5 V** with USB only).

Examples with $V_{\text{supply}} = 8.4\,\text{V}$: $u = 255 \Rightarrow 8.4\,\text{V}$, $u = 128 \Rightarrow 4.2\,\text{V}$, $u = 0 \Rightarrow$ motor stopped. The sign of u determines the direction of rotation.

5 Troubleshooting

✘ Serial port does not appear in MATLAB

- Check that the USB cable is properly connected.
- On Windows: open Device Manager and look for *"Ports (COM & LPT)"*. If a yellow triangle appears, reinstall the CH340 or CP2102 driver.
- On Mac: run `ls /dev/tty.usb*` in Terminal.
- Restart MATLAB after connecting the Arduino.

✘ Motor does not spin

- Verify that the power supply is on and correctly connected.
- Check that the control signal sent from MATLAB is not 0.
- Ensure the motor cables are properly connected to the driver terminals.
- If the signal is very small ($|u| < 30$) the motor may not overcome the dead-zone (static friction). This is normal; it is addressed in Lab 02.

✘ Encoder not responding / position always 0

- Check the 7-pin JST encoder connector: it must be fully seated.
- Open the Arduino IDE Serial Monitor at 115 200 baud: if no data appears, the encoder is not initialising. The firmware prints an error message in that case.
- Verify that the shaft magnet is correctly aligned above the encoder (recommended air gap: 0.5 – 1.5 mm).

✘ Simulink disconnects mid-experiment

- This usually happens when another programme is using the same serial port (e.g. the Arduino IDE Serial Monitor). **Close the Serial Monitor before launching Simulink.**
- If the problem persists, disconnect and reconnect the USB cable and relaunch the model.

✘ Velocity signal is very noisy

- This is expected: velocity is obtained as the position difference divided by T_s, which amplifies quantisation noise from the encoder.
- Solution: apply a first-order low-pass filter in MATLAB/Simulink with time constant $T_f \approx 0.02 - 0.05\,\text{s}$. Covered in Lab 01.

6 Repository and Resources

Resource	Location
Full repository	`github.com/DocMuerde/Motor-CC-Control-Lab`
Arduino firmware	`firmware/arduino/main/main.ino`
MATLAB scripts	`software/matlab/`
Simulink model	`software/matlab/MotorCC_V3.slx`
Lab guide — Lab 01	`labs/lab01/lab01_isa.pdf`
Lab guide — Lab 02	`labs/lab02/lab02_isa.pdf`
PCB schematics	`hardware/pcb/`
3D-printed parts	`hardware/3d-printing/`

■ Note

To clone the full repository:

```
git clone https://github.com/DocMuerde/Motor-CC-Control-Lab.git
```